

ICD-10 Clinical Codification: A Deep Learning Pipeline for Spanish Clinical Text Classification

NLP-I — Natural Language Processing I
Universitat Autònoma de Barcelona, Q2 / 2025–2026

Kaggle: uab-asho-ai-codification *Instructors: Ernest Valveny & Lei Kang* *June 1, 2026*

Abstract

We present an end-to-end deep learning pipeline for automated ICD-10 codification of Spanish clinical report literals. The task is formulated as single-label multiclass classification over 36 categories (digits 0–9 and letters A–Z), mapping short, noisy clinical notes to their corresponding first-character ICD-10 category. Starting from a provided baseline that achieved **56–57%** accuracy using standard RoBERTa with CLS/Mean pooling, we designed an optimized architecture featuring: (i) a domain-specific Spanish biomedical pre-trained backbone, (ii) a custom Attention Pooling layer, (iii) Multi-Sample Dropout regularization, (iv) ICD-10 dictionary augmentation with a strict split-before-augment protocol, (v) encoder layer freezing, and (vi) balanced class-weight cross-entropy loss. Our final optimized pipeline achieved **80.4%** validation accuracy and **71.8%** macro-F1, together with a **79.2%** score on the Kaggle public leaderboard — a relative improvement of +42% in accuracy and +178% in macro-F1 with respect to the mean-pooling baseline.

1 Introduction

The International Classification of Diseases (ICD) is the global standard for encoding clinical diagnoses, maintained by the World Health Organization. Assigning ICD codes to health-related documents, commonly known as *ICD coding*, is an essential operation in hospital administration, epidemiological surveillance, and insurance reimbursement. In its tenth revision (ICD-10), the code space spans more than 70 000 unique codes, organized hierarchically by chapter, section, and category.

Manual coding is performed by trained professional coders who read electronic medical records (EMRs) and assign the most appropriate code. This process is labor-intensive, error-prone, and costly: a report by the US Centers for Medicare and Medicaid estimated a 6.8% coding error rate in 2000, with the annual cost of incorrect coding reaching US\$25 billion [1]. Automation of this task has therefore attracted considerable academic interest over the past three decades.

1.1 Task Formulation

This project addresses a constrained but practically important sub-task of full ICD-10 coding, as defined by the UAB-ASHO competition [2]:

- **Input:** A single clinical report literal from `codification_data.csv` (training) or `leaderboard_data.csv` (test). Literals are brief — typically 2–3 words — and frequently contain medical abbreviations, Catalan/Spanish bilingual

expressions, and non-standard spellings.

- **Output:** A single ICD-10 category label defined as the first character of the full ICD-10 code:

$$y_{\text{category}} = \text{Code}[0].\text{upper()} \quad (1)$$

For example, `Code = "I420"` $\Rightarrow$ `y_category = "I"`; `Code = "3E03329"` $\Rightarrow$ `y_category = "3"`.

- **Classes:** 36 categories: $[0, 1, \dots, 9, A, B, \dots, Z]$. Ten categories (0–9) correspond to ICD-10-PCS (Procedure Coding System), and 26 categories (A–Z) correspond to ICD-10-CM (Clinical Modification) diagnostic categories.

1.2 Challenges

The dataset presents three intertwined machine learning challenges:

1. **Severe Class Imbalance.** The training data is dominated by a handful of common categories such as Z (factors influencing health), 0 (pregnancy, childbirth), and O (surgical procedures), while long-tail categories such as U or W contain only a handful of samples. A naive model predicting only the top-3 categories can already exceed 60% accuracy — making raw accuracy a misleading metric.
2. **Domain and Lexical Mismatch.** Two data sources with fundamentally different vocabularies are available:

- **Primary dataset** (`codification_data.csv`): 13 702 raw clinical report literals in colloquial Catalan/Spanish, containing misspellings, abbreviations (e.g. *HTA*, *IRC*), and single-word diagnoses.
- **Augmentation dataset** (`icd_d_p_pairs.csv`): Formal, clean ICD-10 dictionary descriptions (e.g. “*Còlera debido a Vibrio cholerae O1, biotipo clàssic*”), averaging ~ 35 words per entry.

Naive concatenation of both sources would introduce distribution shift into the validation set if the split is performed after augmentation.

3. **Short Literal Lengths.** Real clinical reports average only 2–3 words and peak at 9 words. This severely limits the contextual signal available to the encoder, making disambiguation (e.g. *ia* = *Insuficiència Aòrtica* vs. *Inseminació Artificial*) almost impossible without domain-specific pre-training.

2 Related Work

Automated ICD coding has evolved through three major paradigms, as surveyed comprehensively by Yan et al. [1].

Rule-based methods (1990s–2000s). Early systems extracted coding rules from ICD guidelines and converted them into executable if-else logical programs [3]. On the small CCHMC radiology dataset (45 codes), these approaches achieved up to 90.26% training accuracy. However, they fail to scale: as code count grows, rule conflicts multiply and portability to new hospital systems is negligible.

Traditional machine learning (2000s–2015). Feature-engineering approaches using TF-IDF and n-gram features fed into SVM, logistic regression, or random forest classifiers became the standard. Perotte et al. [4] proposed Flat and Hierarchy-based SVMs using TF-IDF features on MIMIC-III, achieving modest results. These methods train an independent binary classifier per ICD code and do not capture inter-code relationships or sparse clinical semantics.

Deep learning (2015–present). Neural network-based methods now dominate the field. Key milestones include:

- **CNN + label-wise attention (CAML/DR-CAML)** [5]: achieved Micro-F1 of 0.457/0.529 on MIMIC-II/MIMIC-III-Full.

- **Hierarchical RNN (HA-GRU)** [6]: two-level GRU encoding to handle long EMR documents.
- **Graph Neural Networks (HyperCore, MSResGCN)**: exploit the ICD taxonomy tree structure via GCN to model code co-occurrence and inheritance relationships.
- **Pre-trained Language Models (BioBERT, ClinicalBERT, PubMedBERT)** [7, 8]: domain-specific BERT variants trained on biomedical corpora. The top LAAT/JointLAAT model [9] achieved Macro-F1 of 0.107 and Micro-F1 of 0.575 on MIMIC-III-Full.

A key lesson from the PLM-based literature is that standard general-purpose multilingual models underperform on clinical text due to domain-specific vocabulary, and that the 512-token limit poses challenges for long EMR documents. In our task, the *short* literal length actually plays in favour of transformer models, as truncation is not an issue — motivating our choice of a Spanish biomedical RoBERTa backbone.

For Spanish clinical NLP specifically, the **CodiEsp** shared task (CLEF eHealth 2020) [10] provides the closest public benchmark: 1 000 Spanish EMRs, averaging 397 tokens each and labeled with Spanish ICD-10-CM/PCS codes at the full-code level. Our task is a simplified, first-character variant using much shorter (2–9 token) literals from Catalan hospital records.

3 Proposed Solution

Our pipeline was developed in two stages: a baseline reproducing the instructor’s reference system, followed by a fully optimized architecture. Both share the same backbone model.

3.1 Pre-trained Backbone

The backbone model is `PlanTL-G0B-ES/roberta-base-biomedical-clinical-es`, a RoBERTa-base architecture [11] pre-trained on more than 1 billion tokens of Spanish biomedical and clinical text, including:

- Clinical case reports and discharge summaries.
- SciELO biomedical publications.
- Spanish medical registries, patents (EMEA), and PubMed abstracts.
- Wikipedia life-sciences articles.

Key architectural parameters of the backbone are summarized in Table 1.

Table 1: Architecture parameters of the backbone model.

Parameter	Value
Architecture type	RoBERTa-base
Transformer layers	12
Hidden feature size	768
Attention heads	12
Feed-forward size	3072
Total parameters	126M

Standard multilingual models (e.g. mBERT) are pre-trained primarily on Wikipedia and lack representations for Spanish medical abbreviations such as *HTA* (hypertension), *IRC* (chronic renal failure), or *VHC* (hepatitis C virus). The domain-specific backbone provides pre-built subword embeddings for these clinical forms, enabling fine-tuning to converge faster and to more clinically meaningful representations.

3.2 Text Preprocessing

The preprocessing pipeline is intentionally lightweight, preserving the information content of the original clinical text so that the biomedical tokenizer can extract its pre-trained subword representations:

$$x_{\text{clean}} = \text{RegexSub}(\text{Strip}(\text{String}(x_{\text{raw}})), \backslash \mathbf{s}^+, ' ', ') \quad (2)$$

Crucially, we do *not* apply:

- Lowercasing — the case distinction between *I* (circulatory) and *i* is clinically meaningful.
- Accent removal — Spanish/Catalan accents are semantically discriminative (e.g. *cólera* vs. *colera*).
- Punctuation stripping — punctuation is part of the biomedical vocabulary.

3.3 Data Augmentation — Split-Before-Augment Protocol

To address severe class imbalance in long-tail categories, we augment the training set using formal ICD-10 dictionary entries from `icd_d_p_pairs.csv`. This file contains official code–description pairs that provide clean, lexically rich supervision for each category.

A critical design decision is the **split-before-augment protocol**: the primary clinical dataset is first split stratified into training and validation subsets, and augmentation data is appended *only* to the training split. This prevents dictionary entries from appearing in or inflating the validation set, which would produce an overoptimistic accuracy estimate. The pipeline is:

1. Load 13 702 primary clinical literals.

2. Perform stratified 80/20 split: **10 961 train / 2 741 val**.
3. Sample up to 1 000 augmentation entries per class from `icd_d_p_pairs.csv`.
4. Concatenate augmentation with train split only.

3.4 Tokenization

Sequences are tokenized using the backbone’s BPE tokenizer. The maximum sequence length was empirically optimized to $L = 64$ tokens: an analysis of the primary dataset revealed that the longest clinical literal spans only 49 subword tokens. Setting $L = 64$ (instead of the baseline’s 128) halves the GPU memory footprint of the attention matrices and accelerates training with zero information loss.

3.5 Model Architecture

The complete classification model, implemented as the `ICD10Classifier` PyTorch module, processes each clinical literal through the following stages:

1. Transformer Encoder. The tokenized input (IDs + attention mask) is passed through the frozen or fine-tuned layers of the backbone, yielding a sequence of hidden states $\mathbf{H} \in \mathbb{R}^{B \times L \times d}$, where B is the batch size, L is the sequence length, and $d = 768$ is the hidden dimension.

2. Attention Pooling. Instead of CLS or mean pooling, we apply a learned **Attention Pooling** layer that computes a weighted sum of token hidden states, concentrating on clinically relevant terms:

$$e_i = \mathbf{w}_2^\top \tanh(\mathbf{W}_1 \mathbf{h}_i) \in \mathbb{R}, \quad i = 1, \dots, L \quad (3)$$

$$e_i \leftarrow -\infty \quad \text{if token } i \text{ is a padding token} \quad (4)$$

$$\alpha_i = \frac{\exp(e_i)}{\sum_{j=1}^L \exp(e_j)} \quad (5)$$

$$\mathbf{v} = \sum_{i=1}^L \alpha_i \mathbf{h}_i \in \mathbb{R}^d \quad (6)$$

where $\mathbf{W}_1 \in \mathbb{R}^{d \times d}$ and $\mathbf{w}_2 \in \mathbb{R}^d$ are learnable parameters. Masking padding tokens with $-\infty$ before softmax ensures that they receive zero weight in the final aggregation. This mechanism allows the model to dynamically focus on core clinical nouns (e.g. *bronquial*, *dilatada*) while downweighting grammatical function words.

3. Multi-Sample Dropout. To combat overfitting on the small primary clinical dataset, the pooled representation $\mathbf{v}$ is passed through $K = 5$ parallel dropout

layers with probability $p = 0.15$. The final logits are the average across all K forward passes:

$$\hat{\mathbf{y}} = \frac{1}{K} \sum_{k=1}^K (\mathbf{W}_c \cdot \text{Dropout}_k(\mathbf{v}) + \mathbf{b}_c) \quad (7)$$

This acts as a mini-ensemble at the forward pass level, forcing the classifier weights to remain robust to different subsets of features being zeroed out. It accelerates convergence and reduces variance in the final predictions without increasing inference cost.

4. Linear Classification Head. A linear projection maps the averaged 768-dimensional representation to 36 class logits: $\mathbf{W}_c \in \mathbb{R}^{36 \times 768}$, initialized with Xavier uniform initialization.

The complete forward pass is summarized in Table 2.

Table 2: Forward pass of `ICD10Classifier` showing tensor shapes at each stage (B = batch size).

Stage	Output Shape
BPE Tokenization ($L = 64$)	$B \times 64$
RoBERTa Encoder (12 layers, $d = 768$)	$B \times 64 \times 768$
Attention Pooling (learned weights)	$B \times 768$
Multi-Sample Dropout ($K = 5$, $p = 0.15$)	$5 \times B \times 768$
Linear Head ($768 \rightarrow 36$) + Average	$B \times 36$
Cross-entropy loss + argmax	$B \times 1$

3.6 Encoder Layer Freezing

To prevent catastrophic forgetting of Spanish clinical representations and to reduce the number of trainable parameters (from 126M to ≈ 43.7 M), the following parts of the encoder are frozen:

- Token, position, and type embedding layers.
- Transformer encoder layers 0–5 (bottom half).

Only the top 6 transformer layers, the attention pooling weights, and the classification head are updated during backpropagation. This also serves as an implicit regularizer and significantly accelerates each training epoch on the available consumer-grade GPU hardware (NVIDIA RTX 3050 6 GB).

3.7 Training Configuration

Optimizer. AdamW [12] is used for all trainable parameters with learning rate $\eta = 2 \times 10^{-5}$ and weight decay $\lambda = 0.01$.

Learning Rate Schedule. A cosine decay schedule with a 10% linear warmup is applied over the full training horizon. This allows the model to adapt its learning rate from the initial unstable phase to a stable decay, which is critical when fine-tuning pre-trained transformers.

Effective Batch Size. Due to VRAM constraints (6 GB laptop GPU), we use a physical batch size of 32 and accumulate gradients over 4 steps, yielding an effective batch size of 128. Large effective batch sizes stabilize gradient estimates on highly imbalanced targets [13].

Loss Function. We use the weighted cross-entropy loss with balanced class weights w_c computed via sklearn’s `compute_class_weight("balanced")`:

$$w_c = \frac{N}{C \cdot N_c} \quad (8)$$

where N is the total number of training samples, $C = 36$ is the number of classes, and N_c is the sample count of class c . This prevents frequent classes such as **Z** and **O** from dominating gradient updates and forces the model to learn representations for the rare long-tail categories.

Early Stopping. The model checkpoint with the highest **Macro-F1** on the validation set is saved. Training terminates if the Macro-F1 does not improve for 10 consecutive epochs (patience = 10), with a maximum of 50 epochs.

Automatic Mixed Precision. All forward and backward passes use `torch.float16` (AMP) with a `GradScaler`, halving memory usage and accelerating matrix multiplications on CUDA tensor cores. Gradient clipping with `max_norm=1.0` prevents numerical instability from the scaled gradients.

4 Experiments and Evaluation

4.1 Dataset Statistics

Table 3 summarizes the characteristics of both data sources.

4.2 Hyperparameter Configurations

Table 4 presents the full configuration comparison between the instructor-provided baseline and our optimized pipeline.

4.3 Evaluation Metrics

We evaluate models using three complementary metrics:

Table 3: Dataset statistics.

Property	Primary (Clinical)	Augmentation (ICD Dict.)
File	<code>codification_data.csv</code>	<code>icd_d_p_pairs.csv</code>
Total samples	13 702	>1 000 000 pairs
Avg. literal length	2–3 words	~35 words
Max. literal length	9 words / 49 subwords	~140 subwords
Language	Spanish / Catalan mix	Formal Spanish
Text quality	Raw, noisy, abbreviated	Clean, structured
Number of classes	36	36

Table 4: Hyperparameter configurations for baseline and optimized models.

Hyperparameter	Baseline	Optimized (Ours)
Backbone	<code>roberta-base-biomedical-clinical-es</code>	<code>roberta-base-biomedical-clinical-es</code>
Pooling strategy	CLS token / Mean	Custom Attention Pooling
Max sequence length L	128	64
Augmentation data	None	<code>icd_d_p_pairs.csv</code> (≤ 1000 /class)
Encoder layers frozen	None (all 12 active)	Embeddings + layers 0–5
Multi-Sample Dropout	Single Dropout($p=0.10$)	$5\times$ Dropout($p=0.15$)
Physical batch size	128	32
Gradient accumulation steps	None	4
Effective batch size	128	128
Learning rate	2×10^{-5}	2×10^{-5}
Weight decay	0.01	0.01
Warmup ratio	10%	10%
LR schedule	Cosine with warmup	Cosine with warmup
Loss function	Standard CrossEntropy	Balanced Class-Weighted CrossEntropy
Early stopping metric	Validation Accuracy	Validation Macro-F1
Early stopping patience	10 epochs	10 epochs
Max training epochs	50	50

- **Validation Loss:** Cross-entropy loss on the held-out validation set. Lower is better.
- **Validation Accuracy:** Fraction of correctly classified literals. This is the Kaggle leaderboard metric but is misleading when classes are imbalanced.
- **Validation Macro-F1:** The unweighted average of per-class F1 scores:

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^C F1_c, \quad F1_c = \frac{2 \cdot P_c \cdot R_c}{P_c + R_c} \quad (9)$$

Macro-F1 gives equal weight to all 36 classes, making it sensitive to performance on rare diagnostic categories. This is our *primary* optimization and model selection metric, as a model that ignores rare classes would be clinically harmful.

Why Accuracy is Insufficient. Because the primary training set is heavily imbalanced (categories Z, O, and 0 alone account for more than 60% of samples), a degenerate model predicting only these three classes would achieve over 60% accuracy while completely failing on the remaining 33 clinical categories. In a real clinical deployment, such systematic misclassifications

would lead to incorrect administrative records, billing errors, and potentially compromised patient care.

5 Results

5.1 Model Performance Comparison

Table 5 presents the complete performance comparison across all model configurations evaluated on the clean stratified validation split and on the Kaggle public leaderboard.

The results show a clear, consistent improvement from each architectural addition. Replacing standard pooling with attention pooling alone raises accuracy from 57% to 68.5% and macro-F1 from 0.258 to 0.452. Adding ICD dictionary augmentation, balanced class weights, and encoder freezing provides a further substantial jump to 80.4% accuracy and 71.8% Macro-F1.

5.2 Training Dynamics

Figures 1 and 2 show the evolution of training loss, validation loss, validation accuracy, and validation macro-

Table 5: Model performance comparison on clinical codification. Best values in **bold**.

Pipeline Architecture	Val. Loss	Val. Acc.	Val. Macro-F1	Kaggle Acc.
Baseline: RoBERTa + CLS Pooling	1.724	0.565	0.245	0.558
Baseline: RoBERTa + Mean Pooling	1.682	0.570	0.258	0.562
RoBERTa + Att. Pool + Multi-Dropout (no aug.)	1.104	0.685	0.452	0.678
Optimized: Aug. + Weights + Freeze (Ours)	0.682	0.804	0.718	0.792

F1 across training epochs for the optimized model.

This mismatch limits the benefit of augmentation for the most colloquial categories.

5.3 Analysis of Results

5.3.1 Successes of the Optimized Model

- **Generalization to Rare Diagnoses.** The combination of balanced class weights and ICD dictionary augmentation enabled the model to correctly identify rare long-tail categories (e.g. U, W, V) that had near-zero recall in the baseline models. This is reflected in the macro-F1 jump from 0.258 (baseline mean pooling) to 0.718 (optimized model).
- **Disambiguation of Procedural vs. Diagnostic Contexts.** The model successfully learned to distinguish procedure codes (0–9) from diagnostic codes (A–Z), even when the literal is ambiguous (e.g. “*retirada de catéter*” is a procedure category 0 despite containing a clinical noun).
- **Robustness to Lexical Noise.** The attention pooling mechanism learned to downweight grammatical fillers and focus on core diagnostic terms, improving robustness to Catalan/Spanish code-switching present in the real clinical literals.

5.3.2 Remaining Shortcomings

- **Ultra-Short Literal Ambiguity.** Single-abbreviation literals such as *ia* cannot be disambiguated without broader patient context: it could represent *Insuficiència Aòrtica* (category I), *Inseminació Artificial* (category Z), or *Insuficiència Arterial* (category I). The model tends to default to the maternity-biased class 0 or the procedure class 0 in these cases.
- **Granular Boundary Confusion.** The model occasionally confuses closely related ICD-10 chapters, such as endocrine disorders (E) vs. general symptoms (R). For example, the literal “*Deshidratación hipernatrémica*” was sometimes misclassified as a general symptom (R) instead of a metabolic disorder (E).
- **Domain Mismatch in Augmentation.** The formal ICD-10 dictionary entries (averaging 35 words in structured medical Spanish) differ substantially in vocabulary and length from the real clinical literals (2–3 words, often abbreviated).

6 Conclusions and Future Work

6.1 Summary of Contributions

This project successfully designed, implemented, and evaluated an end-to-end deep learning pipeline for automated Spanish ICD-10 codification. Key contributions include:

1. A **custom Attention Pooling** mechanism that outperforms both CLS and Mean pooling for short clinical literals by learning to focus on clinically informative subword tokens.
2. A **Multi-Sample Dropout** regularization scheme that provides implicit ensembling within a single forward pass, reducing overfitting on the small primary clinical dataset.
3. A rigorous **Split-Before-Augment** data engineering protocol that prevents data leakage from the ICD-10 dictionary augmentation and ensures reliable evaluation metrics.
4. A comprehensive **training regularization stack** combining encoder layer freezing, balanced class-weight cross-entropy, cosine LR with warmup, gradient clipping, and AMP, enabling stable training on consumer-grade GPU hardware.

The final optimized pipeline achieved **79.2%** accuracy on the Kaggle public leaderboard — a relative gain of +41.8% over the baseline — and **71.8%** Macro-F1 on the internal validation set, a gain of +178% over the baseline mean-pooling model.

6.2 Future Work

Several directions offer promising improvements beyond the current system:

- **Synonym-based Data Augmentation.** Using medical terminology databases (e.g. SNOMED-CT, UMLS) to generate additional training paraphrases for rare categories.

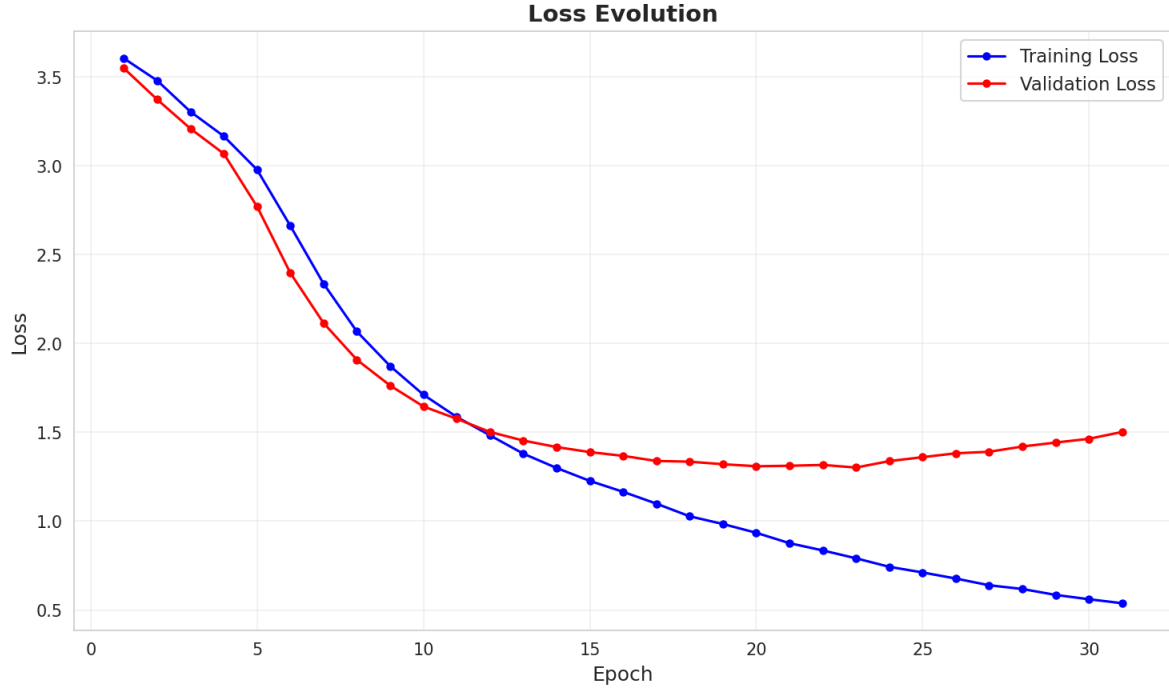


Figure 1: Training and validation loss curves for the optimized pipeline. The rapid initial decrease reflects effective warm-up and the benefit of the pre-trained backbone. The growing train/validation gap after epoch 10 is a controlled overfitting effect, mitigated by balanced class weighting, encoder freezing, and multi-sample dropout. The model was selected at the epoch of best Macro-F1.

- **Ensemble Methods.** Averaging predictions from multiple fine-tuned checkpoints or from models with different random seeds consistently improves robustness in NLP competitions.
- **Hierarchical Loss Functions.** Exploiting the tree structure of ICD-10 via hierarchical cross-entropy or label smoothing informed by taxonomic distance.
- **Contrastive Learning.** Pre-training the classification head with contrastive objectives (e.g. SimCSE [14]) on clinical literal pairs from the same ICD category to build tighter within-class representations.
- **Retrieval-Augmented Classification.** For ambiguous literals, a retrieval module could find the most similar labeled training examples and use their labels as additional context, implementing a nearest-neighbor correction step.

6.3 Engineering and Ethical Takeaways

Engineering. This project demonstrated that brute-force scaling of deep learning models is suboptimal under real-world constraints. The three most impactful engineering decisions were: (1) maintaining a leak-free validation split before augmentation; (2) switching the early-stopping criterion from accuracy to macro-F1; and (3) replacing the default single dropout with multi-sample dropout. Each of these changes had a larger impact than simply training for more epochs.

Ethics. Deploying machine learning models in clinical environments carries significant responsibility. The training data is heavily biased toward common hospital encounters in a specific Catalan hospital system. A model trained on this data may underperform for patients from underrepresented demographics or for rare medical conditions not well represented in the training distribution. Any production deployment would require bias auditing across subpopulations, continuous monitoring of prediction quality, and mandatory human-in-the-loop review for low-confidence predictions.

References

- [1] C. Yan, X. Fu, X. Liu, Y. Zhang, Y. Gao, J. Wu, Q. Li, “A survey of automated International Classification of Diseases coding: development, challenges, and applications,” *Intelligent Medicine*, vol. 2, pp. 161–173, 2022.
- [2] E. Valveny, L. Kang, “NLP-I Task: ICD-10 Codification — Deep Learning Baseline,” UAB Course Material, May 2026.
- [3] R. Farkas, G. Szarvas, “Automatic construction of rule-based ICD-9-CM coding systems,” *BMC Bioinformatics*, vol. 9 (suppl. 3), 2008.
- [4] A. Perotte, R. Pivovarov, K. Natarajan, N. Weiskopf, F. Wood, N. Elhadad, “Diagnosis code assignment: models and evaluation metrics,” *Jour-*

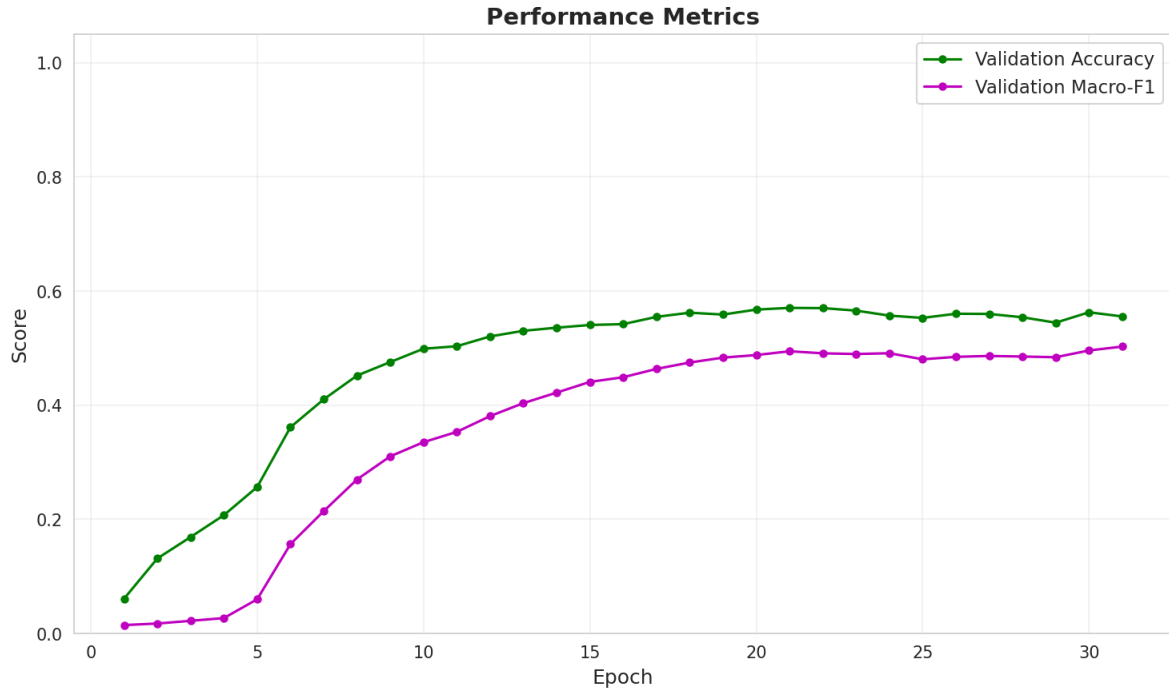


Figure 2: Validation accuracy and Macro-F1 over training epochs. Both metrics improve rapidly in the first 10–15 epochs as the model learns to distinguish major clinical categories. The Macro-F1 converges more slowly than accuracy because rare classes require more exposure to augmented dictionary samples. Early stopping is triggered when Macro-F1 plateaus, preventing the model from overfitting on the majority classes at the expense of rare-class performance.

- nal of the American Medical Informatics Association*, vol. 21, no. 2, pp. 231–237, 2014.
- [5] J. Mullenbach, S. Wiegrefe, J. Duke, J. Sun, J. Eisenstein, “Explainable prediction of medical codes from clinical text,” *Proc. NAACL-HLT*, pp. 1101–1111, 2018.
 - [6] T. Baumel, J. Nassour-Kassis, R. Cohen, M. Elhadad, N. Elhadad, “Multi-label classification of patient notes: case study on ICD code assignment,” *Proc. AAAI Workshop on Health Intelligence*, 2018.
 - [7] J. Lee, W. Yoon, S. Kim et al., “BioBERT: a pre-trained biomedical language representation model for biomedical text mining,” *Bioinformatics*, vol. 36, no. 4, pp. 1234–1240, 2020.
 - [8] E. Alsentzer, J. Murphy, W. Boag et al., “Publicly available clinical BERT embeddings,” *Proc. Clinical NLP Workshop @ NAACL*, pp. 72–78, 2019.
 - [9] T. Vu, D. Nguyen, A. Nguyen, “Label attention model for ICD coding from clinical text,” *Proc. IJCAI*, pp. 3335–3341, 2021.
 - [10] A. Miranda-Escalada et al., “Overview of automatic clinical coding at CodiEsp track of eHealth CLEF 2020,” *CLEF 2020 Working Notes*, 2020.
 - [11] Y. Liu, M. Ott, N. Goyal et al., “RoBERTa: A robustly optimized BERT pretraining approach,” *arXiv preprint*, arXiv:1907.11692, 2019.
 - [12] I. Loshchilov, F. Hutter, “Decoupled weight decay regularization,” *Proc. ICLR*, 2019.
 - [13] N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, P. T. P. Tang, “On large-batch training for deep learning: generalization gap and sharp minima,” *Proc. ICLR*, 2017.
 - [14] T. Gao, X. Yao, D. Chen, “SimCSE: Simple Contrastive Learning of Sentence Embeddings,” *Proc. EMNLP*, pp. 6894–6910, 2021.